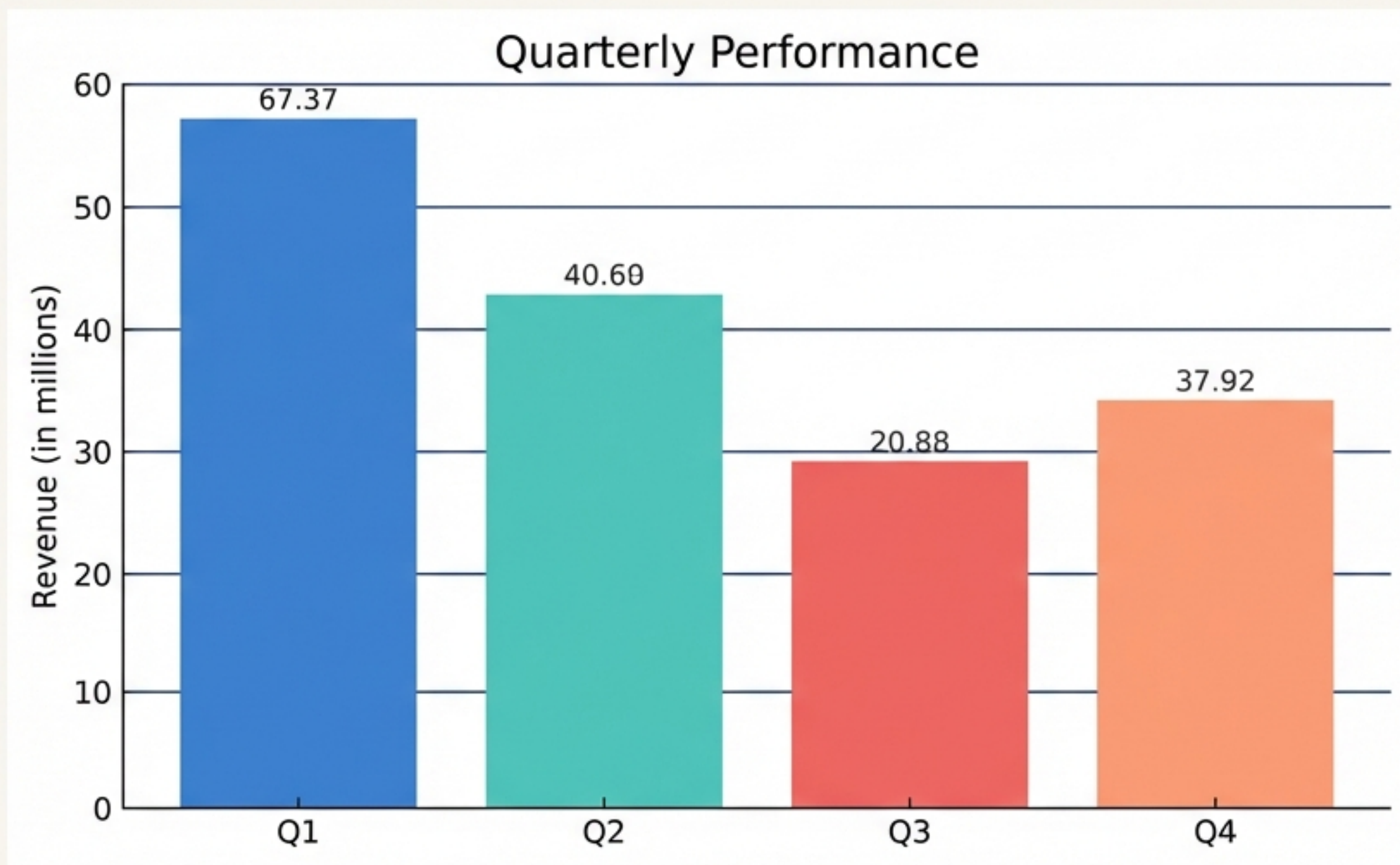


Generating Publication-Ready Visualizations with Diffusion Models

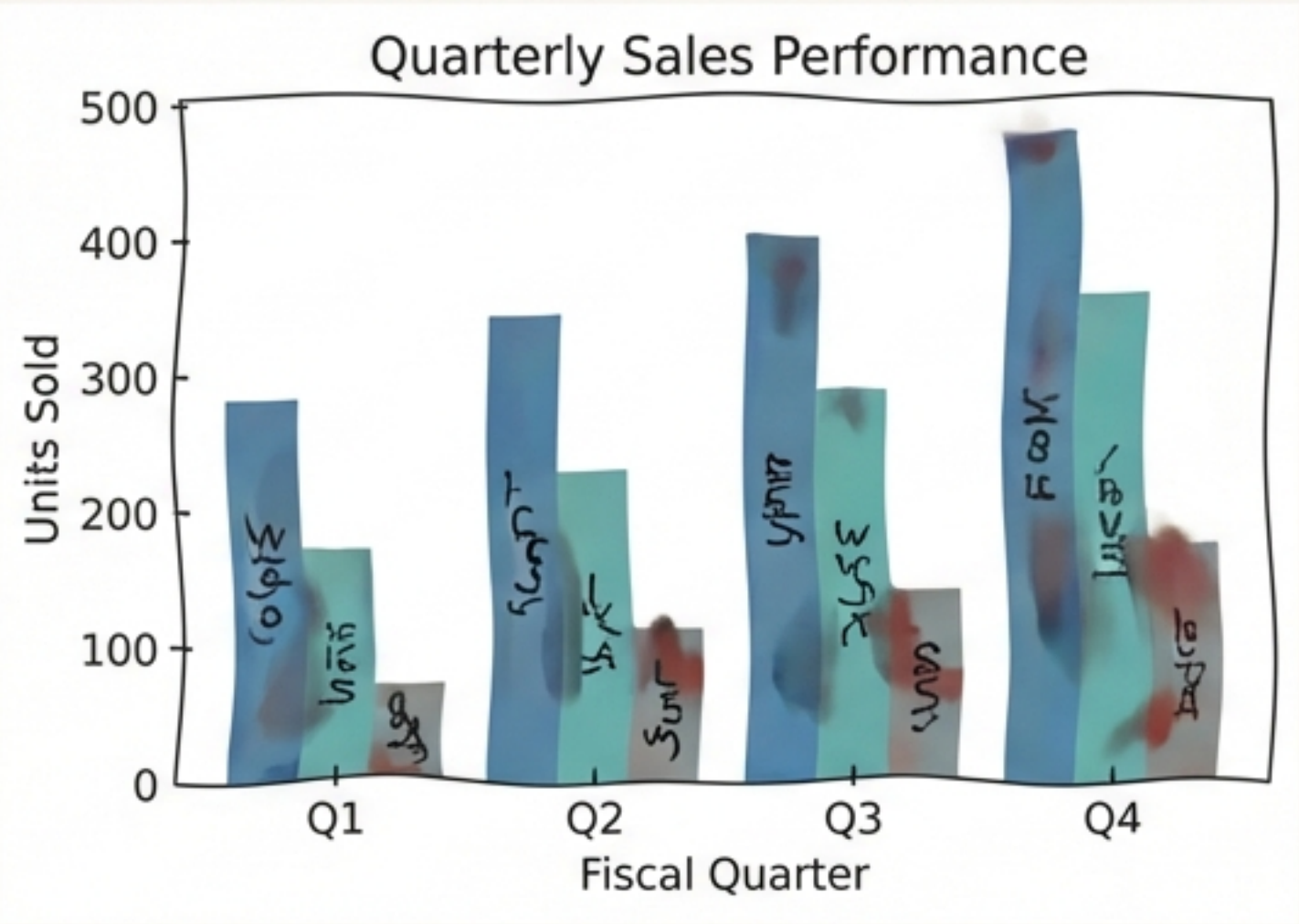
A hybrid approach to fine-tuning for textual clarity and structural integrity.



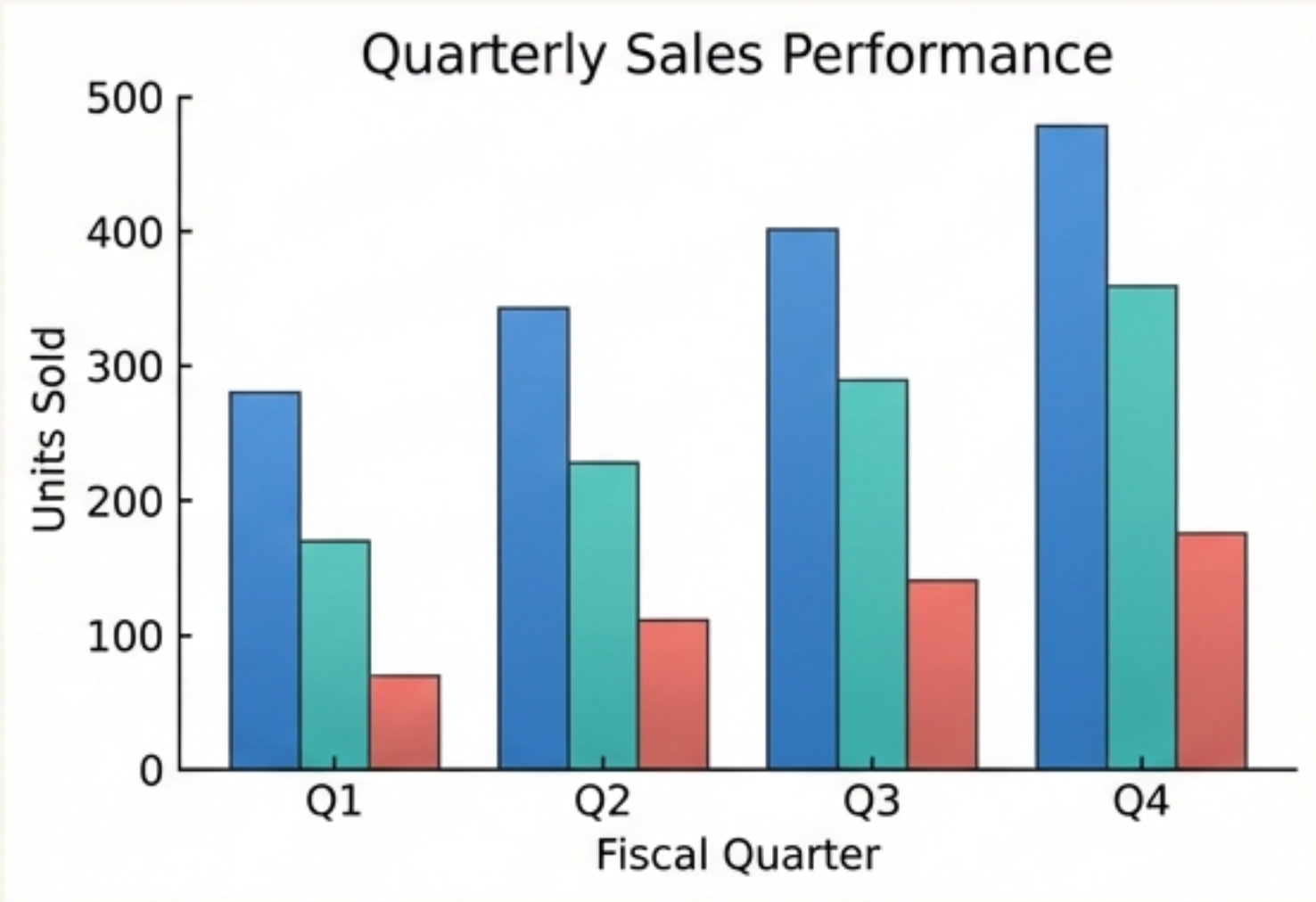
The Challenge: Diffusion Models Struggle with Textual Precision

Standard text-to-image models excel at generating photorealistic and artistic imagery. However, for specialized domains like academic and technical diagrams, they fail on a critical dimension: rendering coherent, legible text. This limitation makes them unsuitable for creating publication-ready charts and figures directly from prompts.

Base Diffusion Model Output



Ground Truth Ideal



Our Solution: A Hybrid of Generative Structure and Programmatic Precision

We combine the structural generation capabilities of a fine-tuned diffusion model with the absolute precision of programmatic text rendering.



1. Generative Foundation

Use a diffusion model to learn and generate the underlying visual structure of various charts (layouts, colors, shapes).



2. Programmatic Overlay

Employ a dedicated text rendering engine to overlay clean, accurate, and perfectly placed labels post-generation.



3. Domain-Specific Data

Fine-tune the model on a custom-built, synthetic dataset of ideal academic visualizations to master the domain's visual language.

The End-to-End Workflow



Stage 1: Synthetic Data Generation

Programmatically generate thousands of clean visualizations (flowcharts, bar charts, etc.) with perfect text overlays.
Key classes:
``AcademicVisualizationGenerator``, ``TextOverlayHelper``.



Stage 2: Efficient Model Fine-Tuning

Use Low-Rank Adaptation (LoRA) to efficiently fine-tune a Stable Diffusion v1.5 model on our synthetic dataset.
Key class:
``AcademicVizModelTrainer``.



Stage 3: Hybrid Inference

Generate a base visualization from a text prompt, then apply the programmatic text overlay for a flawless final output.

Step 1: Crafting a High-Fidelity Synthetic Dataset

The quality of a fine-tuned model is determined by its training data. We constructed a robust pipeline to generate a diverse dataset of pristine academic visualizations, ensuring the model learns from ideal examples.

Part A: Structure Generation

We use `matplotlib` to programmatically create the foundational visual elements of various chart types. This provides a clean, consistent structural base.

```
class AcademicVisualizationGenerator:
    ...
    def generate_dataset(self, ...):
        generators = [
            self.create_flowchart,
            self.create_bar_chart,
            self.create_line_graph,
            self.create_pie_chart,
            self.create_scatter_plot,
            self.create_network_diagram,
            self.create_venn_diagram
        ]
        ...
```

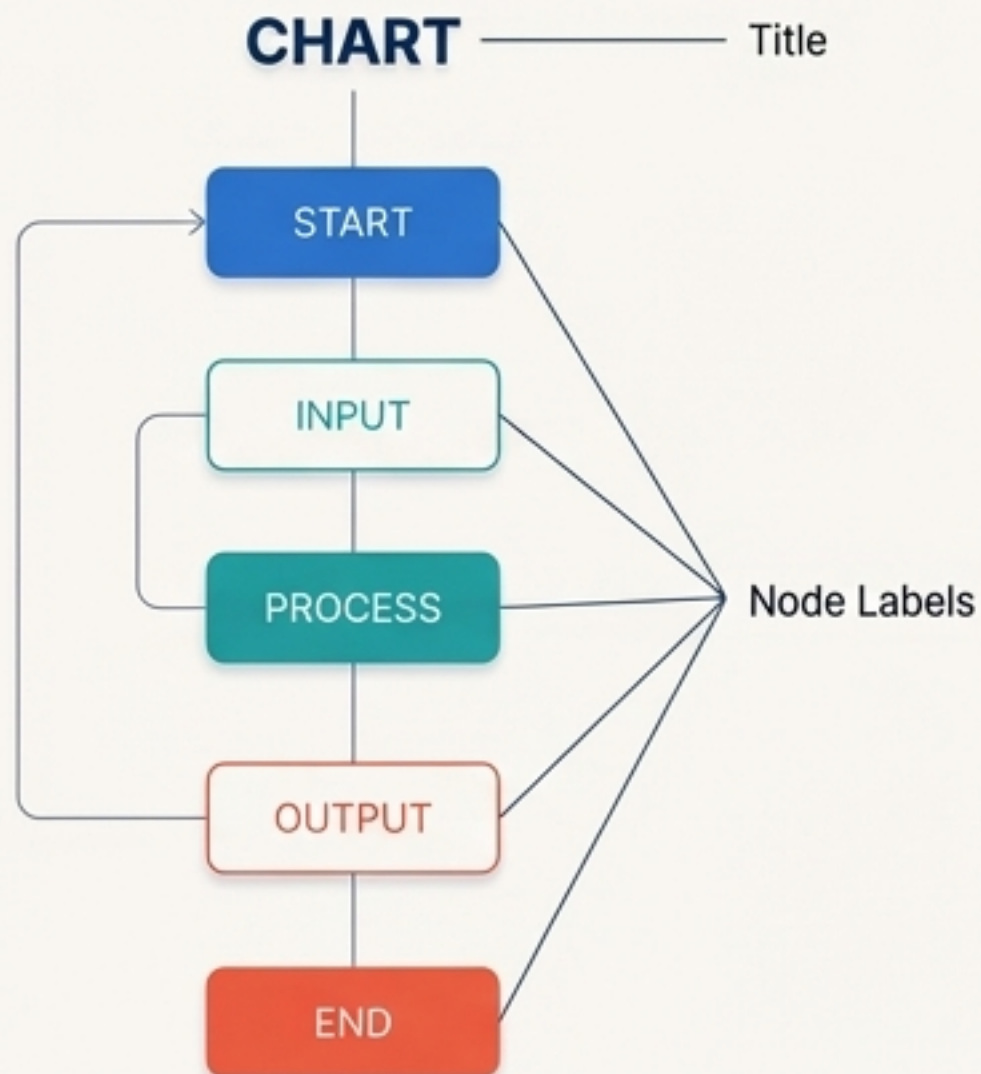
Part B: Text Overlay

We use the Python Imaging Library (PIL) to draw crisp, anti-aliased text directly onto the generated structures. This decouples text rendering from the chart creation, guaranteeing legibility.

```
class TextOverlayHelper:
    ...
    def add_flowchart_labels(self, img: Image.Image) ->
        """Add text labels to flowchart image"""
        ...
```


The Key to Clarity: The `TextOverlayHelper`

This helper class is central to our approach. It encapsulates all logic for rendering text, including font choice, positioning, and adding subtle effects like shadows for readability. It supports a wide range of chart types, each with its own specialized labeling logic.



Snippet 1: Font Loading

```
def __init__(self):
    ...
    font_paths = [
        "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
        "/System/Library/Fonts/Arial.ttf",
        ...
    ]
    for font_path in font_paths:
        if Path(font_path).exists():
            try:
                self.huge_font = ImageFont.truetype(font_path, 48)
                self.large_font = ImageFont.truetype(font_path, 36)
                self.medium_font = ImageFont.truetype(font_path, 28)
                ...
            except:
                continue
    ...
```

Using specific, high-quality fonts for consistency.

Snippet 2: Legibility Logic

```
def draw_text_with_shadow(self, draw, text, position, font, fill='black',
                          shadow_color='white', shadow_offset=3):
    """Draw text with shadow for readability"""
    x, y = position
    # Draw shadow
    draw.text((x + shadow_offset, y + shadow_offset), text, font=font, fill=shadow_color)
    # Draw main text
    draw.text((x, y), text, font=font, fill=fill)
```

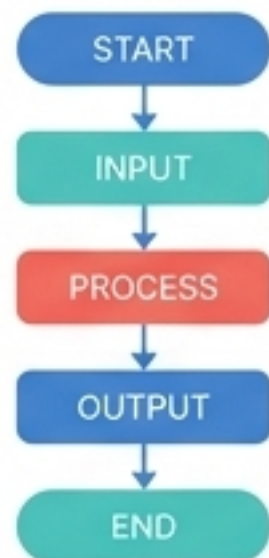
A dedicated function ensures text is always legible against any background.

Snippet 3: Precise Alignment

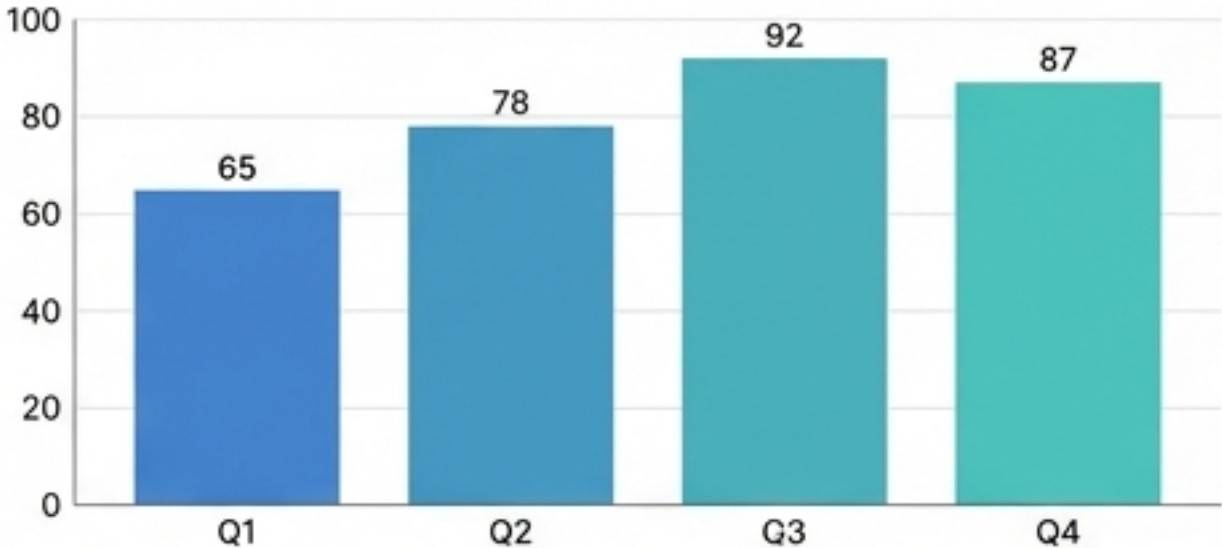
```
def add_flowchart_labels(self):
    ...
    # Title
    title = "CHART"
    bbox = draw.textbbox((0, 0), title, font=self.huge_font)
    text_width = bbox[2] - bbox[0]
    self.draw_text_with_shadow(draw, title, (width // 2 - text_width // 2, 20), self.huge_font, fill='000336', shadow_color='white')
    ...
```

Precise, programmatic positioning for perfect alignment.

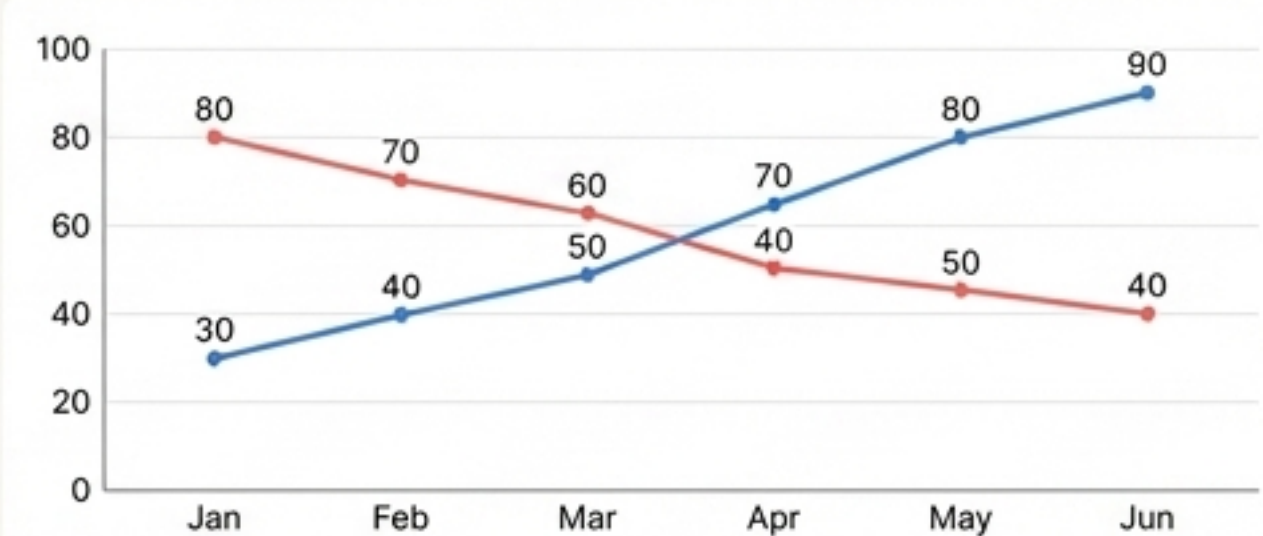
The Result: A Diverse Corpus of Ideal Visualizations



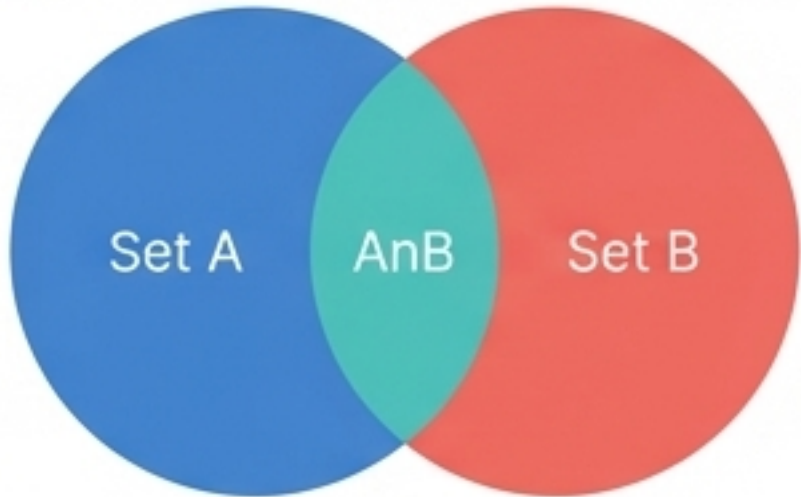
description: "Flowchart with five colored boxes labeled START, INPUT, PROCESS, OUTPUT, END with arrows connecting them vertically."



description: "Bar chart with four bars labeled ('Q1', 'Q2', 'Q3', 'Q4') with values [65, 78, 92, 87]."



description: "Line graph with two trend lines over six months showing upward and downward trends."



description: "Venn diagram with two overlapping circles representing Set A and Set B with intersection labeled AnB."

Total samples generated: 500+

Step 2: Fine-Tuning a Foundation Model

With our high-quality dataset, we can now adapt a powerful, pre-trained diffusion model to our specific domain. We chose ``runwayml/stable-diffusion-v1-5`` as our foundation.

The Challenge

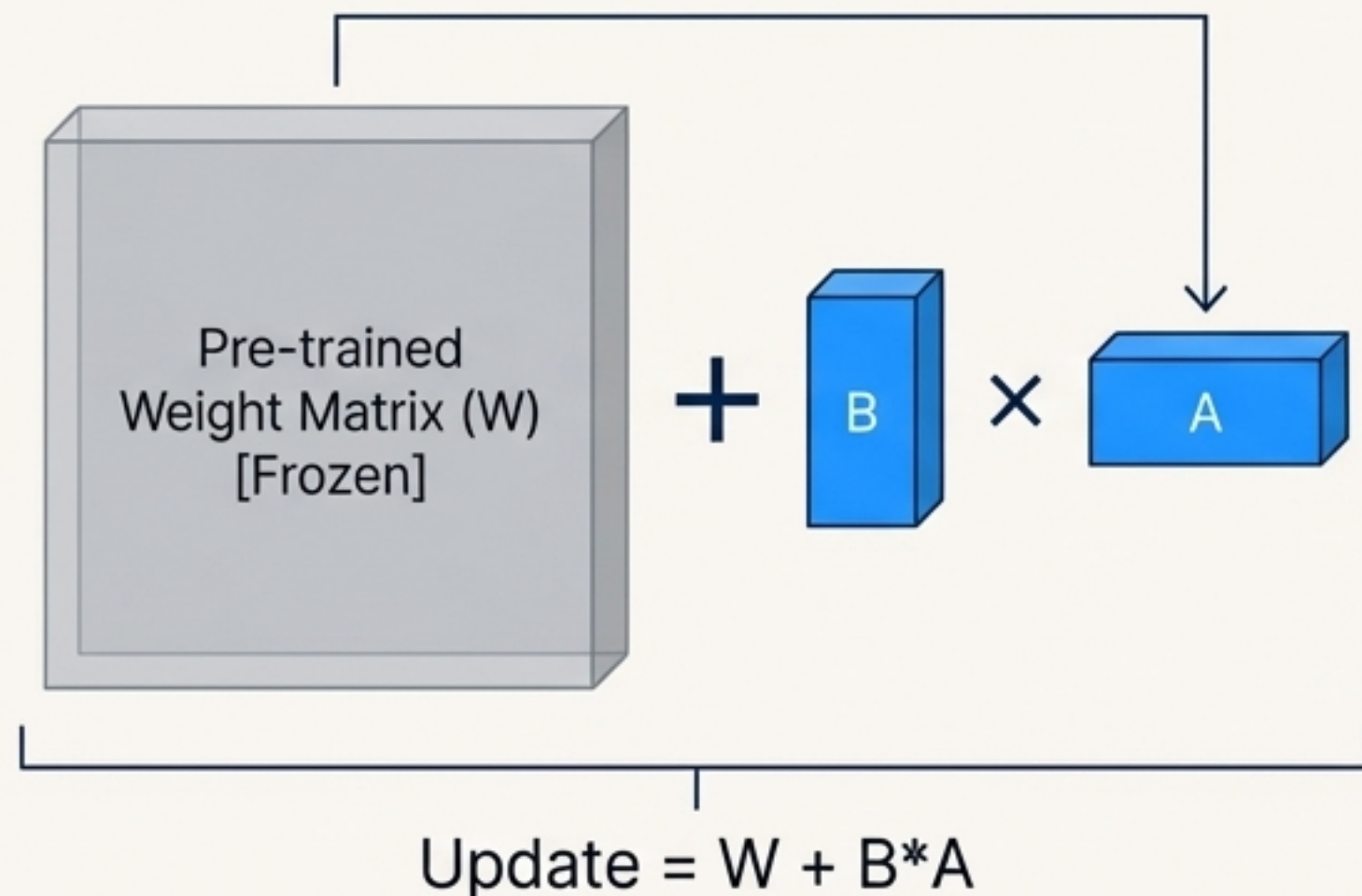
Full fine-tuning of a multi-billion parameter model is computationally expensive and time-consuming. It requires significant GPU resources and risks “catastrophic forgetting” of the model's original capabilities.



Orchestrated by: ``AcademicVizModelTrainer``

The Solution: Efficient Adaptation with LoRA

We use Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning (PEFT) technique. Instead of retraining all model weights, LoRA injects small, trainable "adapter" matrices into the model's key attention layers. This dramatically reduces the number of trainable parameters, leading to faster training and smaller model artifacts.



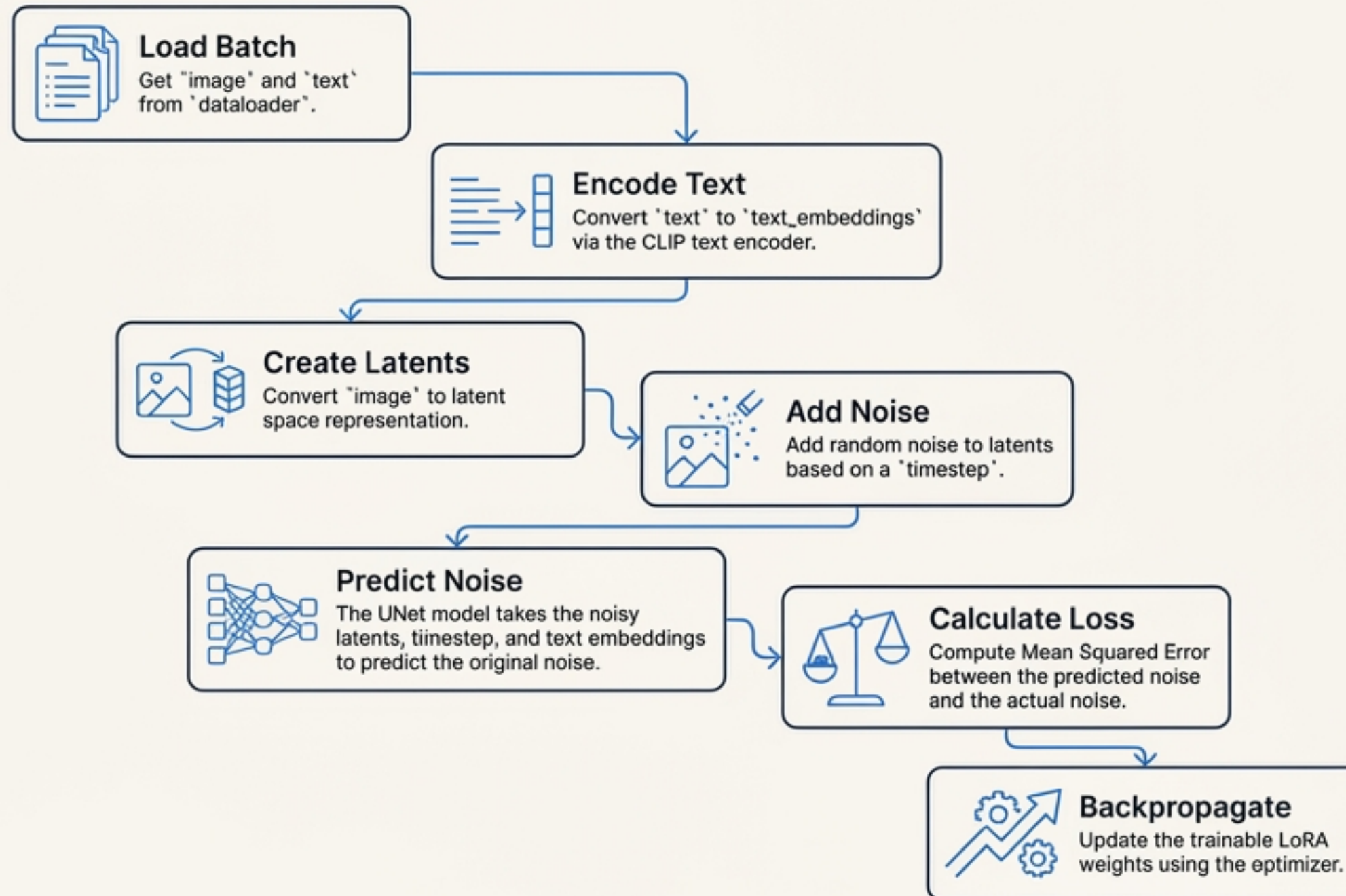
```
from peft import LoraConfig, inject_adapter_in_model

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["to_q", "to_k", "to_v", "to_out.0"],
    lora_dropout=0.1,
    bias="none"
)
inject_adapter_in_model(self.unet, lora_config)
```

We target the query, key, value, and output projection matrices within the UNet's attention blocks.

The Core Training Loop

The model is trained to de-noise an image. During each step, we provide the model with a noisy version of a training image and its text description. The model's task is to predict the original noise, and we update the LoRA weights based on the accuracy of its prediction.



Step 3: Hybrid Inference for Guaranteed Precision

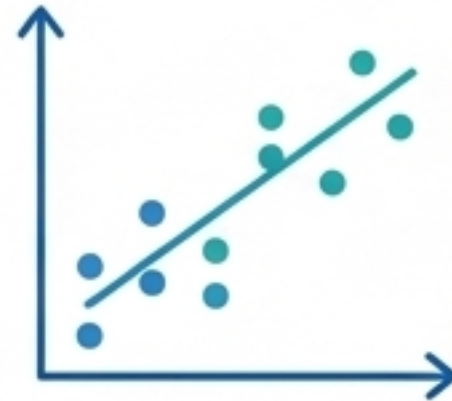
Even with fine-tuning, a diffusion model may still produce minor imperfections in text. To guarantee publication-quality results, we apply our hybrid strategy again during inference.

Step 1: Generate Structure

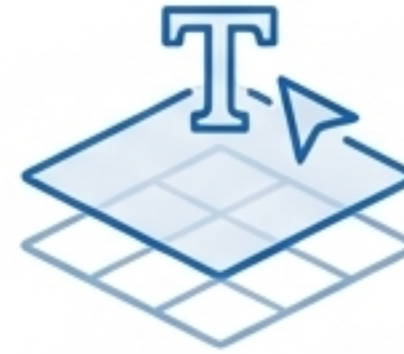
"scatter plot..."



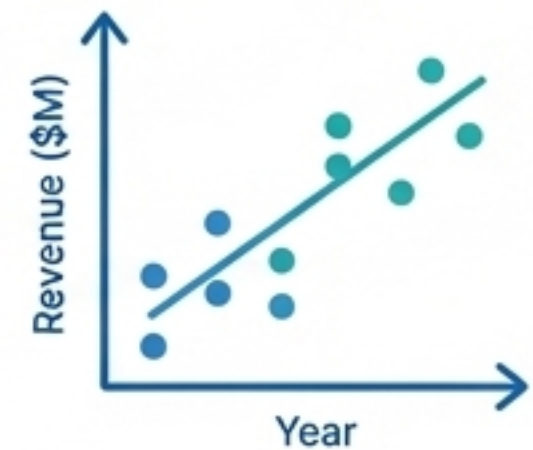
LoRA



Step 2: Overlay Text



Annual Revenue Growth (2020-2024)



The fine-tuned model generates the core visualization—the axes, data points, and trend line—but without any text.

The ``TextOverlayHelper`` programmatically draws the final, pristine text labels onto the generated image. This ensures 100% accuracy and legibility.

Hybrid Inference in Practice

The main execution script orchestrates this two-step process. It first calls the model to generate an image from a test prompt, then uses a conditional block to select the correct `TextOverlayHelper` method based on the requested chart type.

```
# A list of test cases with prompts and annotation types
test_cases = [
    ("A simple flowchart...", "flowchart"),
    ("Bar chart with four colored bars", "bar"),
    ...
]

for prompt, viz_type in test_cases:
    # STEP 1: Generate the base image from the prompt
    image = trainer.generate_and_annotate(
        prompt, annotation_type="none", ...
    )

    # STEP 2: Apply the precise text overlay
    image = trainer.generate_and_annotate(
        prompt, annotation_type=viz_type, image=image
    )

    image.save(...)
```

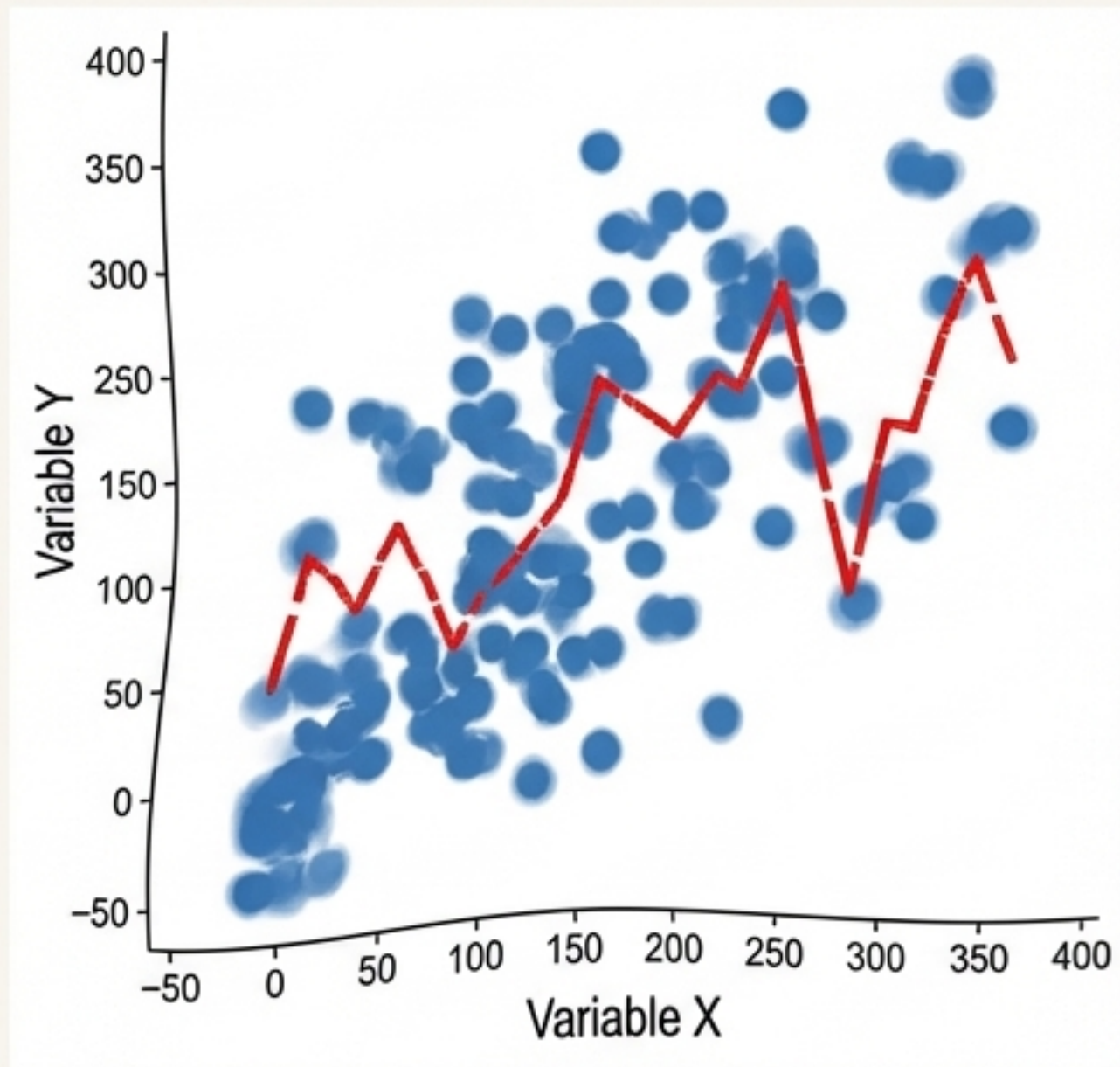
← Step 1: Generate Structure (no text)

← Step 2: Overlay Precision Text

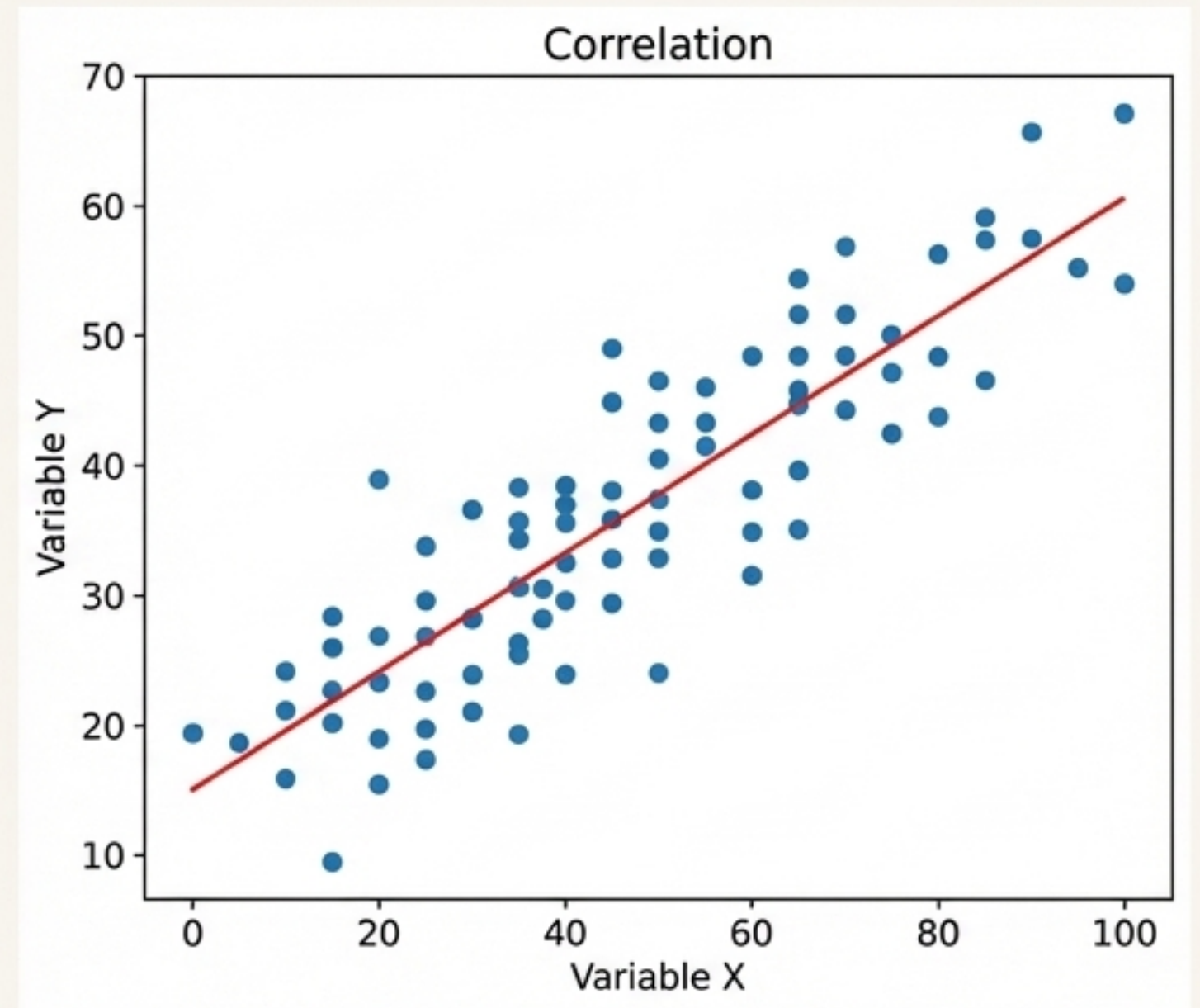
The Transformation

"Scatter plot showing correlation between Variable X and Variable Y with 100 blue points and red trend line."

Base Stable Diffusion v1.5

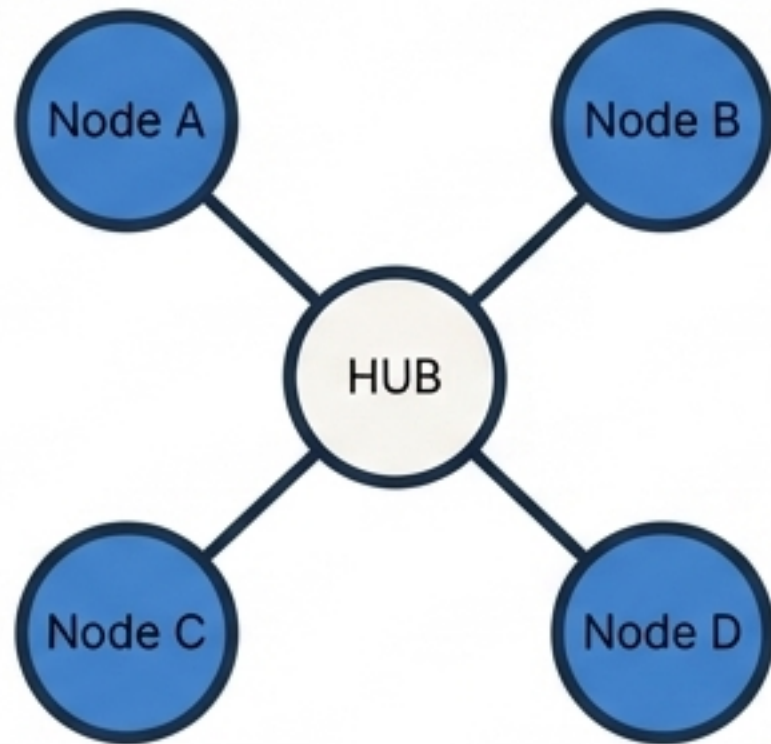


Our Fine-Tuned Hybrid Model

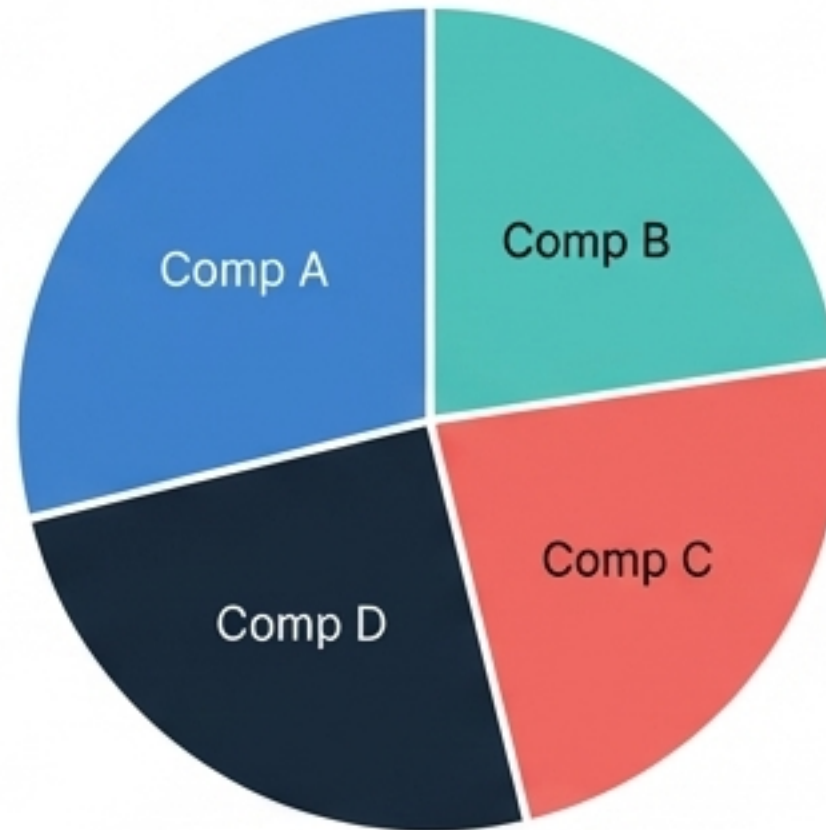


A Showcase of Generated Visualizations

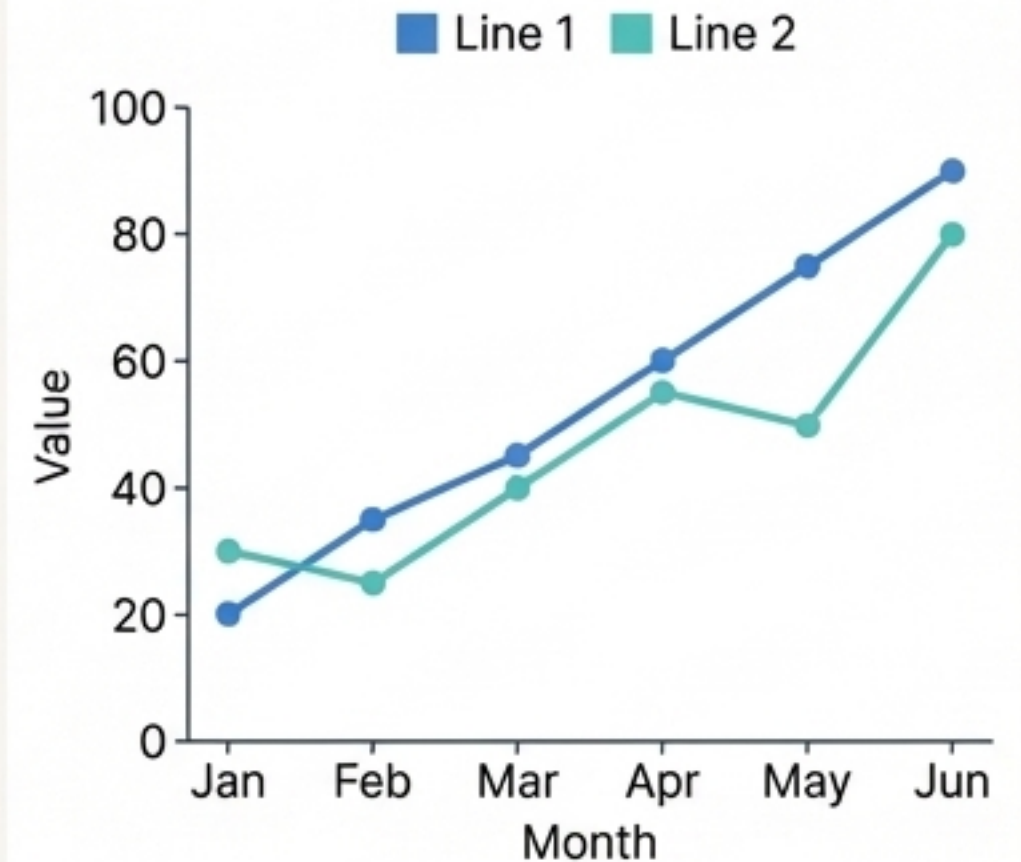
Our method consistently produces high-quality, publication-ready figures across a variety of academic and technical visualization types.



"Network diagram with central HUB node connected to Node A, Node B, Node C, and Node D"



"Pie chart divided into four sections: Comp A, Comp B, Comp C, Comp D"



"Line graph with trend lines for 'Line 1' and 'Line 2' over six months"

Key Takeaways and Future Directions

Key Takeaways

- ✓ **Hybrid is Superior:** For domain-specific tasks requiring precision, a hybrid of generative AI and programmatic logic outperforms a purely generative approach.
- ✓ **Data is Key:** A high-quality, synthetic dataset is crucial for successfully adapting foundation models to specialized domains.
- ✓ **PEFT is Practical:** Techniques like LoRA make fine-tuning large models accessible and efficient without requiring massive computational resources.

Future Directions

- **Expanding the Vocabulary:** Adding more complex chart types (e.g., heatmaps, 3D plots).
- **Interactive Generation:** Allowing users to refine chart elements after the initial generation.
- **Style Adaptation:** Fine-tuning on specific visual chart element after to the comparty.
- **Style Adaptation:** Fine-tuning on specific visual styles (e.g., 'a chart in the style of *The Economist*').